XV6 RACE CONDITIONS

Common Foundation Guide for All Four Team Members

Case Study 14

Tracing how concurrent access to shared kernel data goes wrong without synchronization, and how XV6 spinlocks make it safe.

Read this first as a group. The second PDF divides the deeper work among four members. Verify exact function names, lock names and code against your checked-out XV6 revision. **Repository revision:** _____ (run `git log -1 --format='%h %cd'` in the repo and fill this in). Keep one revision for all code, screenshots and explanations.

1. What is this case study asking us to do?

XV6 runs on several CPUs at the same time. Kernel data structures such as the physical-memory free list and the process table are shared by all CPUs. If two CPUs update the same structure at the same moment without coordination, the structure can be left inconsistent. This case study asks you to explain how that happens, show it, and show how XV6 locks prevent it.

By the end, the group should be able to:

- Define race condition, critical section and mutual exclusion.
- Identify shared kernel data structures in XV6 and the lock that protects each one.
- Explain exactly how a spinlock gives mutual exclusion (including why interrupts are disabled).
- Build step-by-step interleavings of two CPUs that produce a wrong result when the lock is removed.
- Run a real experiment: remove the lock, observe failures (if they occur), restore the lock, compare.
- State honestly what the experiment does and does not prove.

2. The idea in plain language

A **race condition** exists when the result depends on the exact timing of operations by different CPUs. The classic cause is a **read-modify-write** sequence that is not atomic: CPU 0 reads a value, CPU 1 reads the same value, both compute from it, and one update overwrites the other.

```
count++   compiles to three steps:   load  r, count
                                     add   r, r, 1
                                     store count, r

Count = 5.  CPU0: load (5)   CPU1: load (5)   CPU0: store (6)   CPU1: store (6)
Expected 7, got 6.  One update was lost.
```

A **critical section** is the code that touches the shared data. **Mutual exclusion** means only one CPU executes the critical section at a time. XV6 provides it with **spinlocks**: a CPU must acquire the lock before entering and release it after.

3. Core terms every member must know

Race condition: incorrect behaviour that depends on the relative timing of concurrent operations on shared data.

Critical section: a region of code that accesses shared data and must not run on two CPUs at once.

Mutual exclusion: the guarantee that at most one CPU is inside a given critical section.

Atomic operation: an operation that appears to happen all at once. RISC-V provides atomic instructions such as `amoswap`; XV6 builds locks from them.

Spinlock: a lock where a waiting CPU loops (spins) until the lock is free. Defined in `kernel/spinlock.h` and `kernel/spinlock.c`.

Deadlock: CPUs wait on each other forever. Different from a race: a race gives wrong results, a deadlock gives a hang.

Free list: a linked list of free physical pages (`kmem.freelist` in `kernel/kalloc.c`), protected by `kmem.lock`.

Interrupt disabling: `acquire()` turns interrupts off on the local CPU (`push_off()`) so that an interrupt handler cannot try to take a lock the same CPU already holds.

4. Worked example: the physical-memory free list

In `kernel/kalloc.c` each free page stores a pointer to the next free page. `kmem.freelist` points at the first one. The code below is paraphrased from xv6-riscv; **verify against your checkout**.

```
struct run { struct run *next; };
struct { struct spinlock lock; struct run *freelist; } kmem;

void kfree(void *pa) {
  struct run *r;
```

```
  /* ... sanity checks, memset(pa, 1, PGSIZE) ... */
  r = (struct run*)pa;
  acquire(&kmem.lock);
  r->next = kmem.freelist;      // critical section starts
  kmem.freelist = r;            // critical section ends
  release(&kmem.lock);
}

void *kalloc(void) {
  struct run *r;
  acquire(&kmem.lock);
  r = kmem.freelist;            // critical section starts
  if(r) kmem.freelist = r->next;// critical section ends
  release(&kmem.lock);
  if(r) memset((char*)r, 5, PGSIZE);
  return (void*)r;
}
```

A. Race in kalloc() without the lock

Free list: P1 -> P2 -> P3. Two CPUs call `kalloc()` at the same time.

Step	CPU 0	CPU 1	freelist
1	r = kmem.freelist (r = P1)		P1 -> P2 -> P3
2		r = kmem.freelist (r = P1)	P1 -> P2 -> P3
3	kmem.freelist = r->next		P2 -> P3
4		kmem.freelist = r->next	P2 -> P3
Result	returns P1	returns P1	**P1 handed out twice**

Two processes now believe they own the same physical page. One overwrites the other's data, and a later `kfree()` of that page by both would corrupt the list.

B. Race in kfree() without the lock

Free list: F -> G. CPU 0 frees page A, CPU 1 frees page B.

Step	CPU 0	CPU 1	freelist
1	A->next = kmem.freelist (F)		F -> G
2		B->next = kmem.freelist (F)	F -> G
3	kmem.freelist = A		A -> F -> G
4		kmem.freelist = B	B -> F -> G
Result			**Page A is lost** (memory leak)

C. With the lock restored

The second CPU spins in `acquire(&kmem.lock)` until the first calls `release()`. The read-modify-write sequences run one after the other, so both calls see a consistent list: two different pages are returned, and both freed pages stay on the list.

Important: these are *possible* interleavings, not guaranteed results. The race depends on timing, so a lock-free run can pass many times and fail rarely. Report exactly what you observe; do not claim a failure you did not see.

5. How the XV6 spinlock works

Paraphrased from `kernel/spinlock.c`; confirm in your checkout.

```
void acquire(struct spinlock *lk) {
  push_off();                           // disable interrupts (nestable)
  if(holding(lk)) panic("acquire");     // same CPU re-acquiring = bug
  while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
    ;                                   // spin; compiles to RISC-V amoswap
  __sync_synchronize();                 // memory fence
```

```
  lk->cpu = mycpu();
}

void release(struct spinlock *lk) {
  if(!holding(lk)) panic("release");
  lk->cpu = 0;
  __sync_synchronize();                  // finish critical-section writes first
  __sync_lock_release(&lk->locked);      // atomically set locked = 0
  pop_off();                             // re-enable interrupts if outermost
}
```

- **Atomic swap:** `__sync_lock_test_and_set` atomically writes 1 and returns the old value. Only the CPU that saw 0 gets the lock.
- **Memory fences:** `__sync_synchronize()` stops the compiler and CPU from reordering critical-section accesses outside the lock.
- **Interrupts off:** prevents a deadlock where an interrupt handler on the same CPU tries to take a lock already held by interrupted code.
- **Spinlock vs sleep-lock:** spinlocks (short critical sections, busy-wait) vs `kernel/sleeplock.c` (long sections such as disk I/O, process sleeps).

6. Shared kernel data and the lock that protects it

Names below are for **xv6-riscv**. Note: the older x86 XV6 uses a single `ptable.lock`; xv6-riscv has no `ptable` and instead uses a lock per process (`p->lock`). Confirm each in your source.

Shared data	File	Lock	Why it needs mutual exclusion
Free page list `kmem.freelist`	kalloc.c	`kmem.lock`	Avoid double allocation and lost pages
Process table `proc[]` (state, pid, etc.)	proc.c	`p->lock` per process	Avoid two CPUs scheduling the same RUNNABLE process
Next PID counter	proc.c	`pid_lock`	Avoid duplicate PIDs
Parent/child links	proc.c	`wait_lock`	Consistent exit/wait handling
Buffer cache	bio.c	`bcache.lock`	Avoid two buffers for one disk block
Open-file table	file.c	`ftable.lock`	Consistent reference counts
Inode cache	fs.c	`itable.lock`	Consistent inode reference counts
Log	log.c	`log.lock`	Consistent transaction accounting
Tick counter `ticks`	trap.c	`tickslock`	Consistent timer count
Pipe buffer	pipe.c	`pi->lock`	Consistent read/write positions

7. Practical checklist

- Confirm and record the repository revision; keep it fixed.
- Make a backup of `kernel/kalloc.c` (or use git) before editing; you must be able to restore locking exactly.
- Write a small multi-process user program that stresses `kalloc()`/`kfree()` (for example forked children calling `sbrk()` up and down) and checks its own memory for corruption.
- Add it to `UPROGS` in the Makefile following the existing naming convention.
- Run with several CPUs: `make qemu CPUS=4` (confirm the default and syntax in your Makefile).
- Run the test many times with the lock, then without it, then with it again. Record every run, including runs that pass.
- Capture real screenshots of commands and output; keep the exact source used.
- Trace functions and record file and function names; use line numbers only from your own source.
- Compare expected vs observed and list what was not tested.

8. What the experiment proves and does not prove

If failures appear after removing the lock, that **demonstrates a race exists** in that code path. If no failure appears, that does **not** prove the code is safe; the race is timing-dependent and may need more runs, more CPUs or a longer critical section. Runs that pass *with* the lock are evidence, not a proof of correctness. The experiment covers one structure (the free list), not every lock in XV6.

9. Questions all four members should answer

Question	Answer
What is a race condition?	Wrong behaviour that depends on the timing of concurrent access to shared data.
What is a critical section?	Code that accesses shared data and must not run on two CPUs at once.
Why does `kalloc()` need a lock?	Reading `freelist` and updating it is a read-modify-write; two CPUs could take the same page.
What goes wrong in `kfree()` without a lock?	Both CPUs link to the same old head; one freed page is lost.
How does a spinlock guarantee exclusion?	An atomic swap lets only one CPU change `locked` from 0 to 1; others spin.
Why disable interrupts in `acquire()`?	To avoid deadlock if an interrupt handler on the same CPU wants a held lock.
Why release the lock before `memset` in `kalloc()`?	The page is already removed from the list, so only the caller uses it; holding the lock longer just wastes time.
Why might the bug not appear every run?	It needs two CPUs to hit the tiny window at the same moment.
Does passing tests prove correctness?	No; it only shows no failure was observed in those runs.
Race vs deadlock?	A race gives wrong results; a deadlock hangs.

10. Evidence for the report/PPT

- Diagram of two CPUs, the free list and `kmem.lock`.
- Source map: file, function, responsibility, lock used.
- Interleaving timelines for `kalloc()` and `kfree()` (and the scheduler if used).
- Screenshots: the modified (lock removed) code, the test run output, and the restored code.
- Before vs after results table with every run listed.
- Limitations paragraph: timing-dependence, number of runs, what was not tested.

Version and reference notes

Record your revision above. Exact functions, lock names and Makefile syntax must be checked against that checkout. Do not copy line numbers from another version, and never fabricate output, panics, screenshots or counts.

References

- xv6-riscv source: https://github.com/mit-pdos/xv6-riscv
- xv6 book (RISC-V edition), Chapter 6: Locking; MIT 6.1810 / 6.S081 course materials: https://pdos.csail.mit.edu/6.828/
- Follow your department's citation style and list only sources you actually consulted.

All four should read this guide. Each member leads one deeper section, but everyone must understand the shared example, the experiment, and the lock mechanism.